Practical No. 2

Title: Sorting of an array using selection and bubble sort.

Problem Statement:

Consider the telephone book database of N clients. Make use of a hash table implementation to quickly look up a client's telephone number. Make use of linear probing, double hashing and quadratic collision handling techniques.

Aim:

Write C/C++ program to the first year percentage of students in an array and display top five scores. Write function for sorting array of floating point numbers in ascending order using -

- Selection Sort
- Bubble Sort.

Objectives:

- To study concept of Sorting.
- To understand implementation of various sorting algorithms like Selection Sort and Bubble Sort.

Prerequisites:

- Knowledge of C and C++ Programming.
- Knowledge of Arrays.

Outcomes:

- Implement use of Array.
- Implement various sorting algorithms.

Software & Hardware requirements:

- Open Source C/C++ Programming tools like G++/GCC or Eclipse.
- 64-bit Open source Linux or its derivative.

Theory Concept:

Sorting

Sorting means arranging something in a specific order. In Data Structures, Sorting is arranging data elements in a data structure in ascending or descending order. To sort an element in a given array, there are some popular algorithms available:

1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Merge Sort
5. Quick Sort
6. Heap Sort
7. Radix Sort
8. Bucket Sort

Selection Sort

This sorting algorithm is an in-place comparison based algorithm in which the list is divided into two parts, sorted part at left end and unsorted part at right end. Initially sorted part is empty and unsorted part is entire list. Smallest element is selected from the unsorted array and swapped with the leftmost element and that element becomes part of sorted array. This process continues moving unsorted array boundary by one element to the right. This algorithm is not suitable for large data sets as its average and worst case complexity are of $\mathbf{O(n^2)}$ where n are no. of items.

Example- Consider the following array for Selection Sort. For the first position in the sorted list, the whole list is scanned sequentially. The first position where 29 is stored presently, search the whole list and find that 13 is the lowest value. So replace 29 with 13. After one iteration 13, which happens to be the minimum value in the list, appears in the first position of the sorted list. For the second position, where 72 is residing, start scanning the rest of the list in a linear manner and find that 29 is the second lowest value in the list and it should appear at the second place. Swap these values. After two iterations, two least values are positioned at the beginning in a sorted manner. The same process is applied to the rest of the items in the array -

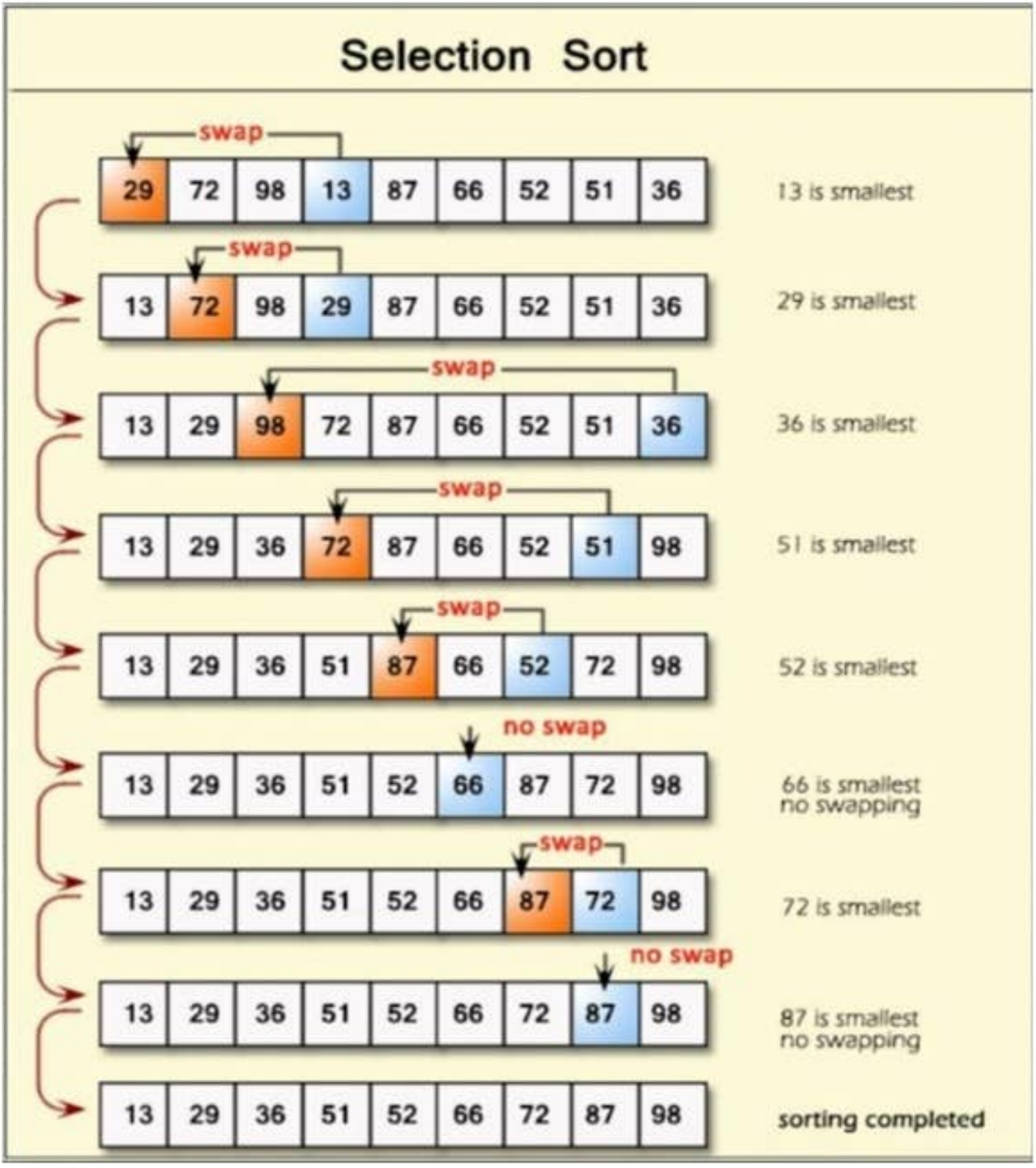


Fig. Selection Sort

Bubble Sort

Bubble sort is a simple sorting algorithm. This sorting algorithm is comparison based algorithm in which each pair of adjacent elements is compared and elements are swapped if they are not in order. This algorithm is not suitable for large data sets as its average and worst case complexity are of $\mathbf{O(n^2)}$ where n are no. of items.

Example- Consider the following array for Bubble Sort. Bubble sort starts with very first two elements, comparing them to check which one is greater. In this case, value 1 is smaller than 5, so swap them. Next, compare 5 with 4. It is found that 4 is smaller than 5 and these two values must be swapped. Next compare 5 and 2. 2 is smaller than 5. So swap them. Next compare 5 and 8. They both are in already sorted positions. After one iteration, array is not sorted. Use this array in next iteration and repeat above sorting process as -

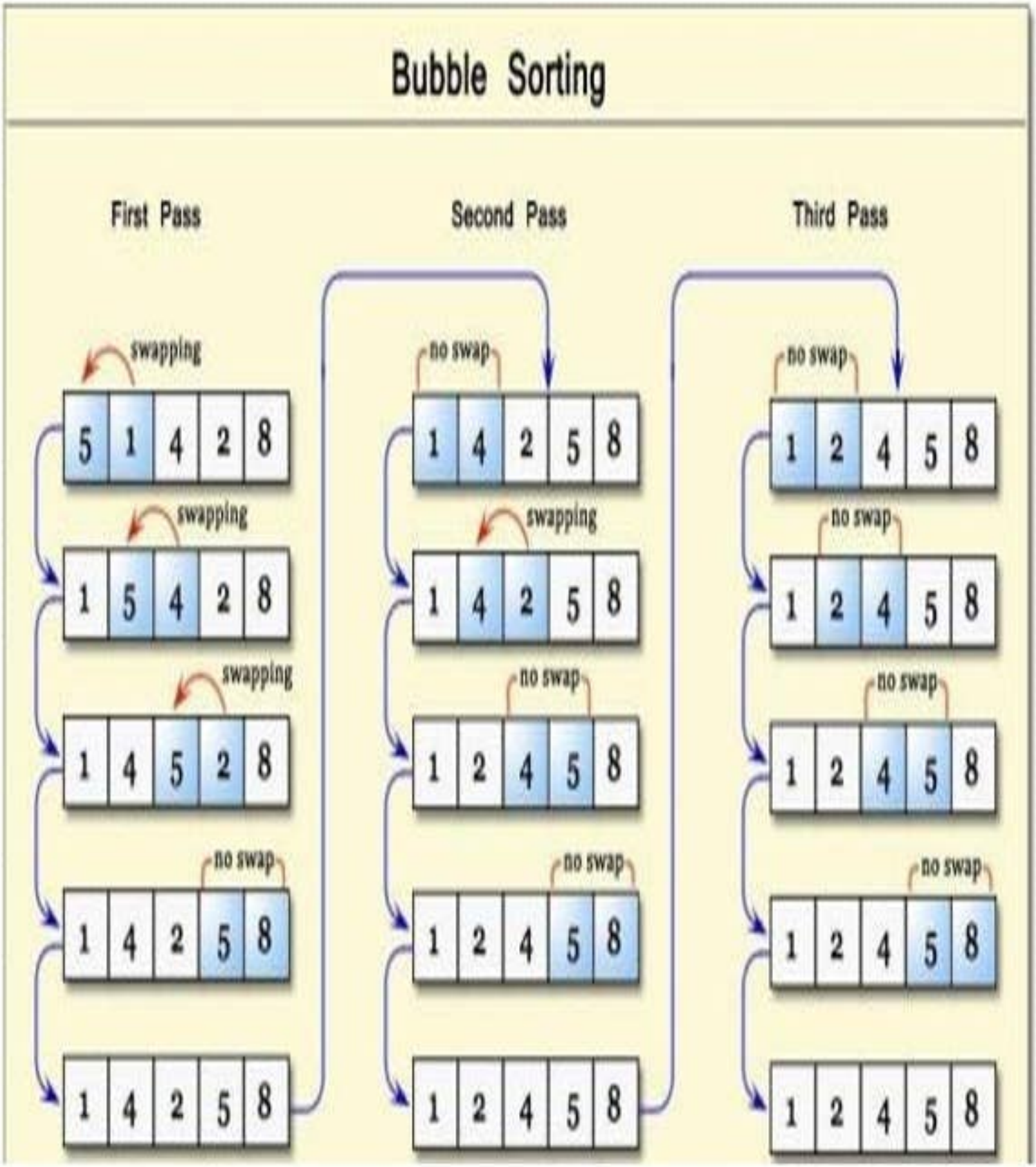


Fig. Bubble Sort

Algorithm:

Below is the complete algorithm for Selection Sort:

Let arr[0..n-1] be the input array to be sorted in ascending order.

1. Start from the first element (index i = 0) and repeat until i < n - 1:
2. Set the current index i as the minimum: min = i.
3. Iterate from j = i + 1 to n - 1:
   If arr[j] < arr[min], update min = j.
4. After the inner loop ends, swap arr[i] and arr[min].
5. Repeat the process for the next position (I = I + 1) until the entire array is sorted.

Flowchart:

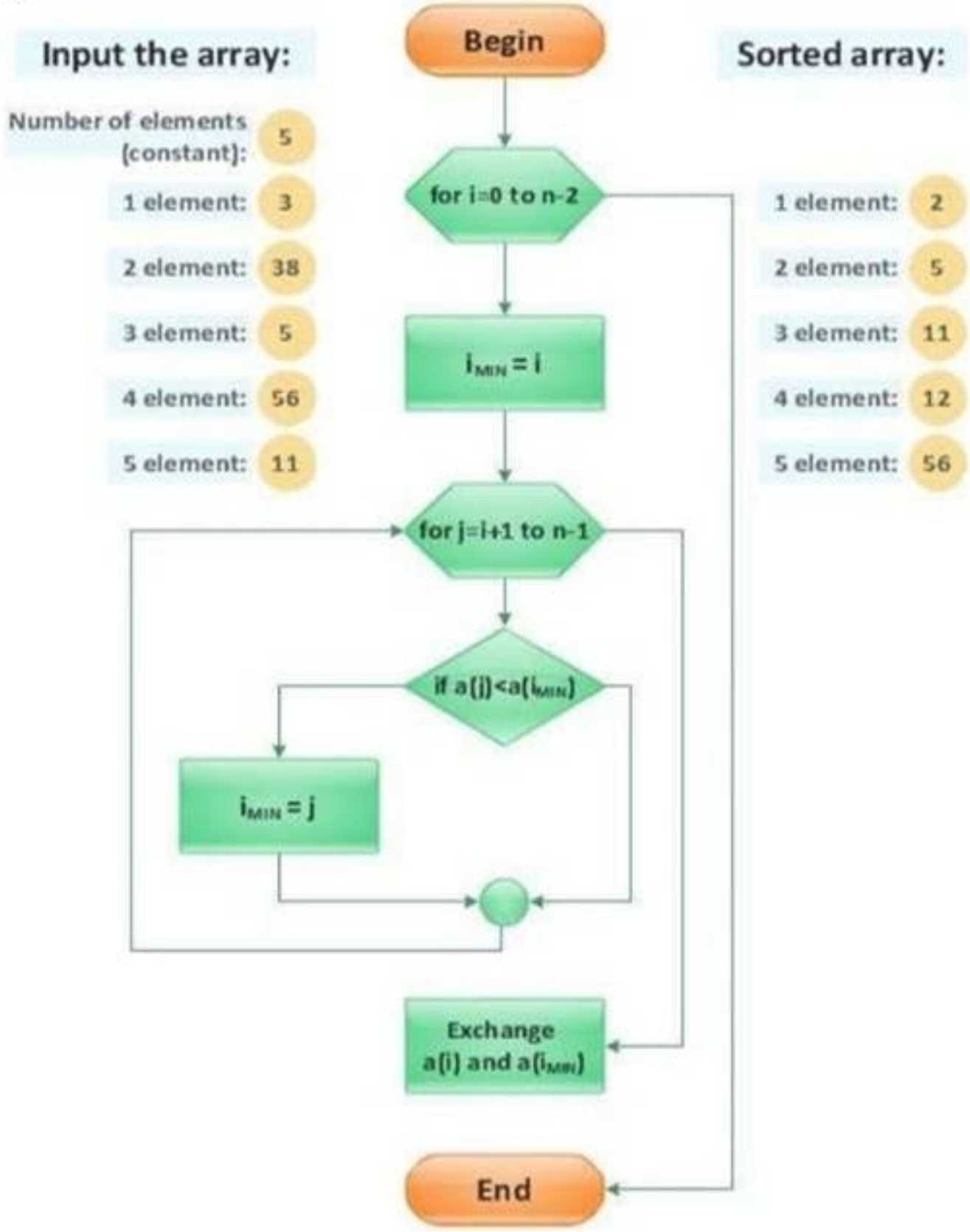


Fig. Flowchart for Selection Sort

Algorithm:

Below is the complete algorithm for Bubble Sort:

Let arr[0..n-1] be the input array to be sorted in ascending order.

1. Repeat the following steps for i = 0 to n - 2:
2. For each pass, compare adjacent elements from index j = 0 to n - i - 2:
   If arr[j] > arr[j + 1], swap arr[j] and arr[j + 1].
3. At the end of each pass, the largest unsorted element "bubbles up" to its correct position.
4. Repeat the process until the array is fully sorted.

Flowchart:

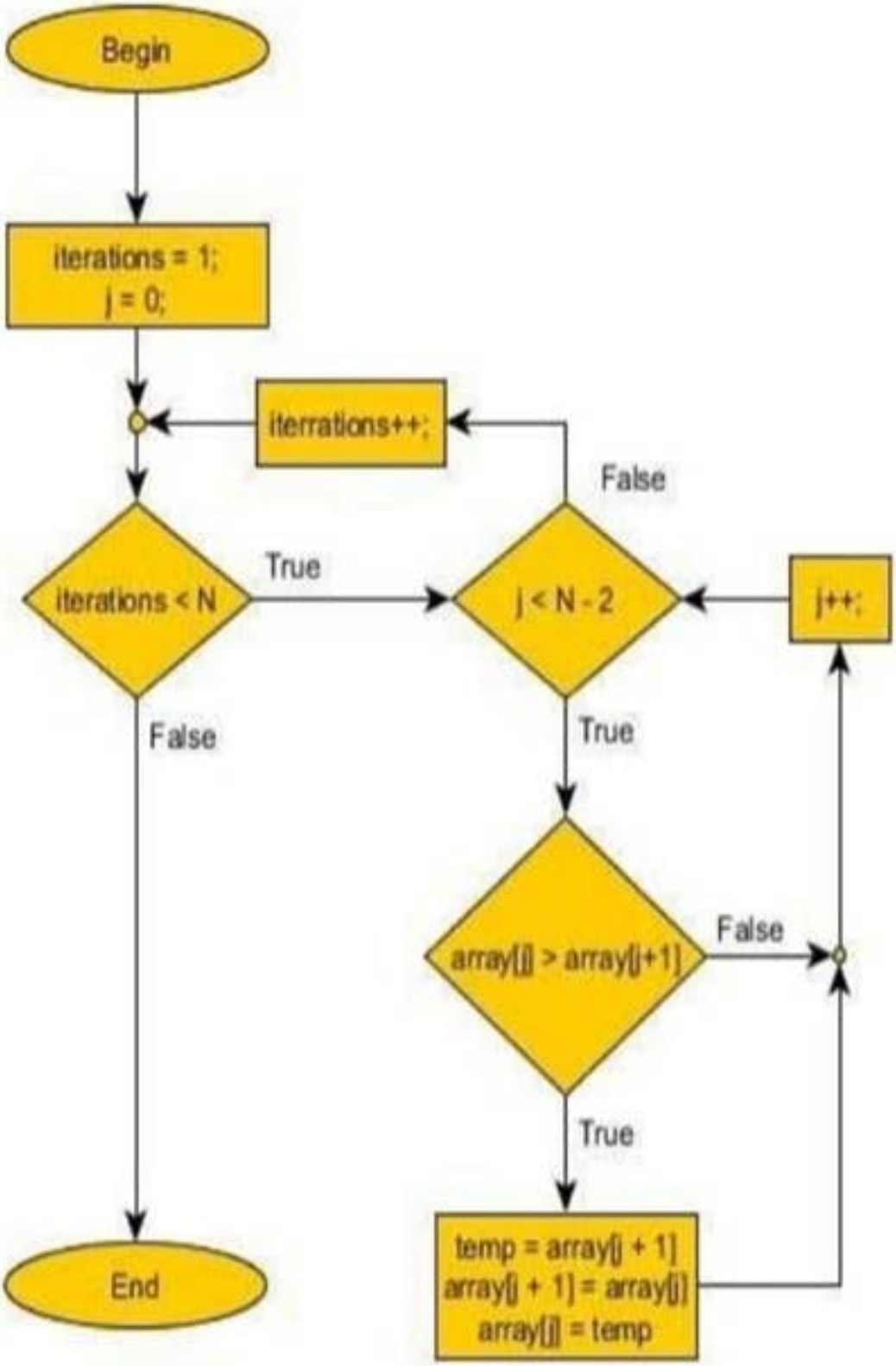


Fig. Flowchart for Bubble Sort

Input: Size of array Elements of array and Student Percentages.
Output: Result based on Selection Sort and Bubble Sort on First Year Student Percentages who attended training Program and display top 5 scores.

Conclusion: Selection Sort and Bubble Sort were studied, understood and implemented on program to store the first year percentage of students in an array using their respective floating point number functions in ascending order and top 5 scores were displayed successfully.